# Supporting Information for:

Bloch-McConnell Simulation Study – Comparison of Various CEST Simulation Frameworks

Markus Huemer[1,2,3], and the BMsim Challenge participants

[1]Institute of Biomedical Imaging, Graz University of Technology, Graz, Austria
[2]Athinoula A. Martinos Center for Biomedical Imaging, Department of Radiology, Massachusetts General Hospital, Charlestown, USA
[3]Department of Radiology, Harvard Medical School, Boston, USA

October 30, 2026

This Supporting Information provides additional implementation details for the participating simulation submissions referenced in the main manuscript, a note on RF frequency-offset handling during interpulse delays, a comparison of interpolation methods for the shaped RF pulse in Case 5, and numerical precision and time-step convergence checks.

## 1 Additional description of simulations

**S01 – M. Zaiss – Pulseq-CEST**
The submission used the MATLAB implementation of Pulseq-CEST [1], which combines a Pulseq-based sequence parser with a C++ Bloch–McConnell simulation core. Pool parameters were read from the YAML model files provided with the challenge, and saturation schemes were simulated from the supplied Pulseq `.seq` files. Magnetization evolution was computed sequentially for each sequence block (RF, delays, and gradients). For piecewise-constant time intervals, the affine Bloch–McConnell system was propagated using a Padé approximation to the matrix exponential. Pools were assembled dynamically from the YAML specification, including separate water, CEST, NOE, and full xyz MT compartments. Shaped RF pulses were resampled according to the maximum number of samples defined in the model file. The Pulseq-CEST code and documentation are available at `https://pulseq-cest.github.io/`.

**S02 – P. Schuenke – BMCTool**
BMCTool is a purely Python-based Bloch–McConnell simulation framework for CEST and related multi-pool exchange experiments. Simulations are driven by Pulseq `.seq` files (parsed via PyPulseq), which provide the full event timing (RF, gradients, delays, ADCs), while scanner settings and pool parameters are specified in YAML configuration files. The magnetization evolution is computed block-by-block: RF events are optionally resampled to a user-defined maximum number of samples and applied piecewise; delays are simulated as free precession/relaxation; and spoiler gradients are modeled as complete transverse spoiling. For each time interval, the affine Bloch–McConnell system $\dot{\mathbf{M}} = \mathbf{AM} + \mathbf{c}$ is propagated using a Padé approximation to the matrix exponential (with the steady-state shift computed from $\mathbf{A}^{-1}\mathbf{c}$ via a pseudoinverse), enabling stable simulation of arbitrary Pulseq-defined irradiation schemes.

**S03 & S04 – M.Huemer & R.Stollberger and M.Huemer & R.Stollberger 2**
Simulation based on work published in [2], written in MATLAB. The second submission does not use the operator splitting proposed in [2], as a check on the influence this has on the simulation results. As described in the paper, a predefined constant time-step is used, and all offsets are

solved in parallel. This can lead to memory problems and limit accuracy with long saturation times and many offsets. In S03 in each case, the timestep was chosen such that all available RAM (64GB) of the test workstation was used. The timesteps were: [1e-4, 1e-5, 1e-5, 5e-8, 2e-7, 1e-5, 1e-5, 1e-7]. The simulation in S04 does not solve all offsets in parallel, therefore the memory demand is much lower and shorter time increments are possible, this can increase performance (visible in Case 3), but leads to longer computation times. The simulation and derivatives are used in [3][4] and the simulation source code is available in the data to reproduce these results `https://doi.org/10.5281/zenodo.15768062`. Scripts to reproduce the simulation results of S03 can also be found here: `https://gitlab.tugraz.at/ibi/mrirecon/miscellaneous/bmc-simulation-matlab`

**S06 – N. Vladimirov & O. Perlman - open-py-cest-mrf**
The simulator was built with a C++ backend, a Pulseq sequence specification, and a Python front end that supports parallel computation. The backend closely follows the Pulseq-cest implementation and provides an interface to facilitate MRF simulations, streamlining parameter modifications. The Bloch-McConnell equations are solved in an iterative manner using the Pade approximation for matrix exponentiation, with a SWIG-based wrapper that enables Python integration. The Python front end is tailored for MRF simulations, using BMCTool definitions to set up and modify parameters on the fly while efficiently handling C++ function calls [5].

**S09 – D. L. Longo & F. Ramdhane**
Simulation based on the Bloch-McConnell work published in [6], written in Matlab by adapting the code available at `https://github.com/cest-sources/BM_sim_fit`. The BM equations have been solved analytically.

**S12 – H.Zhang & J.Huang - CESTsimu**
CESTsimu is an open-source GUI-based CEST simulation tool, written in MATLAB. It is compatible with Pulseq-CEST, able to simulate 1D/2D CEST under $B_0/B_1$ inhomogeneities. The simulation solver is implemented through numerical iteration of the Bloch-McConnell equations. RF events are defined using cell arrays, where each cell represent either a continuous wave or a shaped pulse. The solver processes each RF event sequentially. The time step is specified within each pulse cell. For each time interval, the evolution is modeled in matrix form as $dM/dt = AM + c$, and the equations are solved using two built-in MATLAB functions: `mldivide()` and `expm()`. The solver code along with its demo are available at: `https://github.com/JianpanHuang/BMsim_challenge`.

**S14 – V. Buchegger, M. Juschitz, N. Scholand & M. Uecker – BART**
The BART submission used the Bloch–McConnell simulation implementation in the Berkeley Advanced Reconstruction Toolbox (BART). The BART toolbox is a free and open-source image–reconstruction framework for Computational Magnetic Resonance Imaging [7]. The Bloch–McConnell simulation is written in C and solves the Bloch–McConnell equations using a Runge–Kutta–Dormand–Prince scheme [8]. The main submission (S14) used single-precision floating point arithmetic; an additional variant (S14b) was provided using double precision. The source code is available at `https://codeberg.org/mrirecon/bart`.

**S15 – M. Quach – CESTSim.jl**
CESTSim.jl provides a Julia-based implementation for Bloch-McConnell simulations, with flexible choices of ODE solvers afforded by DifferentialEquations.jl. For the submissions to this challenge, the ODE solver of choice was a 5th-order Rosenbrock-Wanner method [9]. To demonstrate the impact of the choice of solver, an additional submission derived from using a 2/3rd-order Rosenbrock method (*Rosenbrock23()*) was provided to Case 2 (submission S15b in

Figure 2 of the main manuscript). For 5-pool cases, it was assumed that all solute pools underwent exchange with water and water only (i.e., no cross-exchange between non-water pools). The relative tolerance for cases 1-4 (cw) and cases 5-8 (pulsed) were $10^{-3}$ and $10^{-5}$, respectively. The default absolute tolerance of $10^{-6}$ was used for all cases. For cases 5-8, each RF pulse was interpolated and scaled using a constant (0th-order) B-spline, and inter-pulse delays and spoiling delays were simulated as free induction decay events under the presence of exchange. In general, CESTSim.jl supports multi-pool simulations and the construction of complex saturation and imaging sequence protocols (and flexible interpolation options) for simulation. Other conditions such as $\delta\omega$ shift, spoiling, and incomplete recovery can also be incorporated into the simulations.

**S16 – F. Zimmermann et al. – MRpro**
MRpro [10] is an open-source PyTorch-based framework for MR image reconstruction, quantitative parameter estimation, and deep learning. Its Bloch–McConnell simulator is GPU-accelerated and fully differentiable with respect to sequence and tissue parameters. It propagates the affine system $\dot{\mathbf{M}} = \mathbf{AM} + \mathbf{c}$ via matrix-exponential evolution for each sequence block, falling back to an augmented homogeneous formulation when systems are singular. The simulator supports multi-pool exchange, static off-resonance, and relative $B_1$ inhomogeneity through modular blocks including constant RF, piecewise-shaped RF, delays, gradients with isochromat dephasing, spoiling, and readout. For piecewise-shaped pulses, the evolution is computed sample-by-sample with chunked propagation and activation checkpointing to reduce memory use, including in the long pulse-train benchmark cases 6 and 7.

**S17 – O. Cohen**
The simulator was implemented in pyTorch for differentiability and efficient GPU-usage. A helper script was used to interface between the YAML configuration files and the simulator parameters. Bloch-McConnell equations are solved using the pyTorch matrix exponential (`torch.matrix_exp()`) and `torch.linalg.solve()` for the steady-state term. The simulator supports multi-pool exchange for arbitrary number of pools and flexible acquisitions using user-defined acquisition schedules.

**S19 – P. Parnianpour & E. P. Pioro**
The BM equations ($dM/dt = A \cdot M + b$) were solved analytically using the matrix exponential: M(t) = expm$(A \cdot t) \cdot (M_0 - M_{ss}) + M_{ss}$ where $M_{ss} = -A^{-1}b$ is the steady state (via `numpy.linalg.lstsq`). The matrix exponential uses `scipy.linalg.expm` (Al-Mohy & Higham 2009, scaling-and-squaring, Padé order selected automatically based on matrix norm up to m=13). No homogeneous augmentation was used. For shaped pulses (cases 5–7), the Gaussian envelope was sampled into 200 steps matching the .seq file. Case 8 uses block pulses with a 100$\mu$s interpulse delay. For pulse trains (cases 6–7: 36 pulses), the full-pulse propagator was precomputed once per offset and applied 36 times, reducing expm calls by 36×.

## 2 RF frequency offset during interpulse delays

This section describes a possible implementation error that can lead to the second cluster of submissions in Case 6 (Figure 5 of the main manuscript). This error was first noted by Huabin Zhang and Jianpan Huang and this section is based on their observations and analysis.

In pulse-train simulations, the RF carrier offset $\omega_{\mathrm{RF}}$ must be treated consistently during interpulse delays $t_{\mathrm{delay}}$. When $B_1 = 0$, only relaxation and exchange occur, and the rotating frame should revert to the Larmor frame, i.e. $\omega_{\mathrm{RF}} = 0$. If a solver sets the RF amplitude to zero but keeps a static non-zero $\omega_{\mathrm{RF}}$ throughout the delay, the off-resonance term remains in the system matrix and the magnetization continues to precess as if the saturation offset were still applied.

With an off-resonance RF field in the $xy$-plane, the Bloch equation for the magnetization $\vec{M}$ can be written

$$\begin{aligned} \frac{d\vec{M}}{dt} &= \gamma\vec{M} \times \vec{B} \\ &= \gamma\vec{M} \times \Big[\hat{x}B_{1x}(t) + \hat{y}B_{1y}(t) + \hat{z}\Big(B_0 - \frac{\omega_{\mathrm{RF}}}{\gamma}\Big)\Big] \\ &= \gamma\vec{M} \times \big[\hat{x}B_{1x}(t) + \hat{y}B_{1y}(t) + \hat{z}\,\Delta\Omega\big]. \end{aligned} \tag{1}$$

During $t_{\mathrm{delay}}$ the RF amplitude is zero, so $\omega_{\mathrm{RF}}$ should be reset to zero and $\Delta\Omega$ reduces to the chemical-shift offset of each pool in the water-resonant frame. A typical implementation error is to zero $B_1$ but leave $\omega_{\mathrm{RF}}$ at the saturation offset for the entire pulse train, for example by building a static chemical-shift matrix once per offset and reusing it for both RF events and delays:

```
dw = dw_pool - dw_rf * (max(pulse(:,1)) > 0);  % reset omega_RF if B1 == 0
dw = dw_pool - dw_rf;                          % keep omega_RF static
```

Case 6 $Z$-spectra formed distinct groups, with one cluster about 0.026 from a reference implementation in the main cluster and another cluster about 0.17 away. This choice was tested by repeating Case 6 (2-pool model, 36 Gaussian pulses) with and without resetting $\omega_{\mathrm{RF}}$ during $t_{\mathrm{delay}}$ (Figure 1). Resetting $\omega_{\mathrm{RF}}$ reproduced the main cluster (maximum point-wise $|Z|$ difference 0.026 from the reference). Keeping $\omega_{\mathrm{RF}}$ static reproduced the second cluster (maximum difference 0.17 from the reference, and only 0.001 from an example submission in that cluster). The discrepancy is nearly an order of magnitude in some spectral regions, especially near the water resonance and the CEST peak, as the phase evolution leads to different sidebands in the $Z$-spectrum.

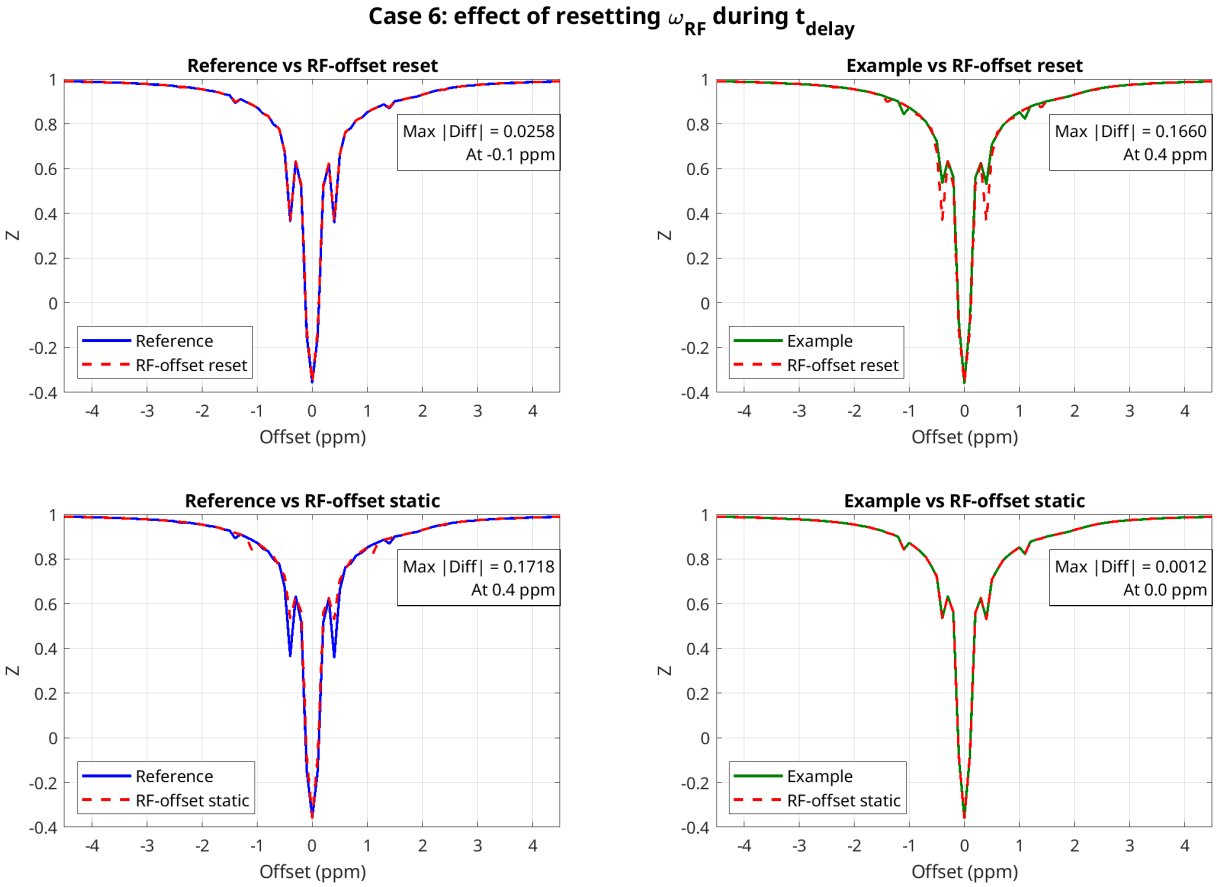



Figure 1: Case 6 $Z$-spectra with and without resetting the RF frequency offset during interpulse delays. Top left: a reference submission from the main cluster versus a solver with $\omega_{\mathrm{RF}}$ reset ($\max|\Delta Z| = 0.026$). Top right: an example submission from the second cluster versus the same reset solver ($\max|\Delta Z| = 0.17$). Bottom left: the reference versus a solver that keeps $\omega_{\mathrm{RF}}$ static ($\max|\Delta Z| = 0.17$). Bottom right: the example submission versus the static-offset solver ($\max|\Delta Z| = 0.001$).

The same pattern is a possible implementation error behind one of the grouped behaviors in Case 6. Different implementations may also lead to similar deviations, and this does not imply that every submission outside the main cluster is affected by this specific error.

Furthermore, Mara Quach noted that padding the pulse shape before interpolation can affect the results together with this $\omega_{\mathrm{RF}}$-reset error, as the interpolation can change the effective duration of the interpulse delay. This is illustrated in Figure 2 using the CESTSim.jl simulation (S15), which shows different padding and time grid choices for the same Gaussian pulse, and the spectra and difference to the Matlab pulseq implementation (S01, marked M.Zaiss in the figure) for each choice.

The top row shows the shape padded with zeros at the beginning and end of the pulse, leading to slightly longer interpulse delays than the original pulse, and a time of half the interpolation raster time, where the RF amplitude is zero but $\omega_{\mathrm{RF}}$ is still set to the saturation offset (at the start of each pulse). The second row shows the same shape padded with $B_1 = 0$ only at the end of the pulse, leading to the same additional delay but no time at the start of the pulse where $\omega_{\mathrm{RF}}$ is still set to the saturation offset. With this padding scheme, the original pulse values are now in the center of the interpolated time grid. The first two rows show very similar spectra and the smallest differences to S01.

The third row shows the same shape padded with zeros only at the end of the pulse and an additional delay time of half the interpolation raster time. The pulse train is equivalent in

timings and RF amplitudes to row one, but $\delta_{\mathrm{RF}}$ is strictly reset to zero for all points where $B_1 = 0$. This leads to a significantly larger deviation from the reference spectrum. The last row uses the same padding as the first row, but without resetting $\omega_{\mathrm{RF}}$ during interpulse delays. This leads to the largest deviation from the reference spectrum, as shown in the previous figure.

The same implementation method can therefore lead to very different spectra depending on the interpolation time grid and padding scheme, showing the intricate nature of the problem and the need for careful implementation and reporting of the simulation details.

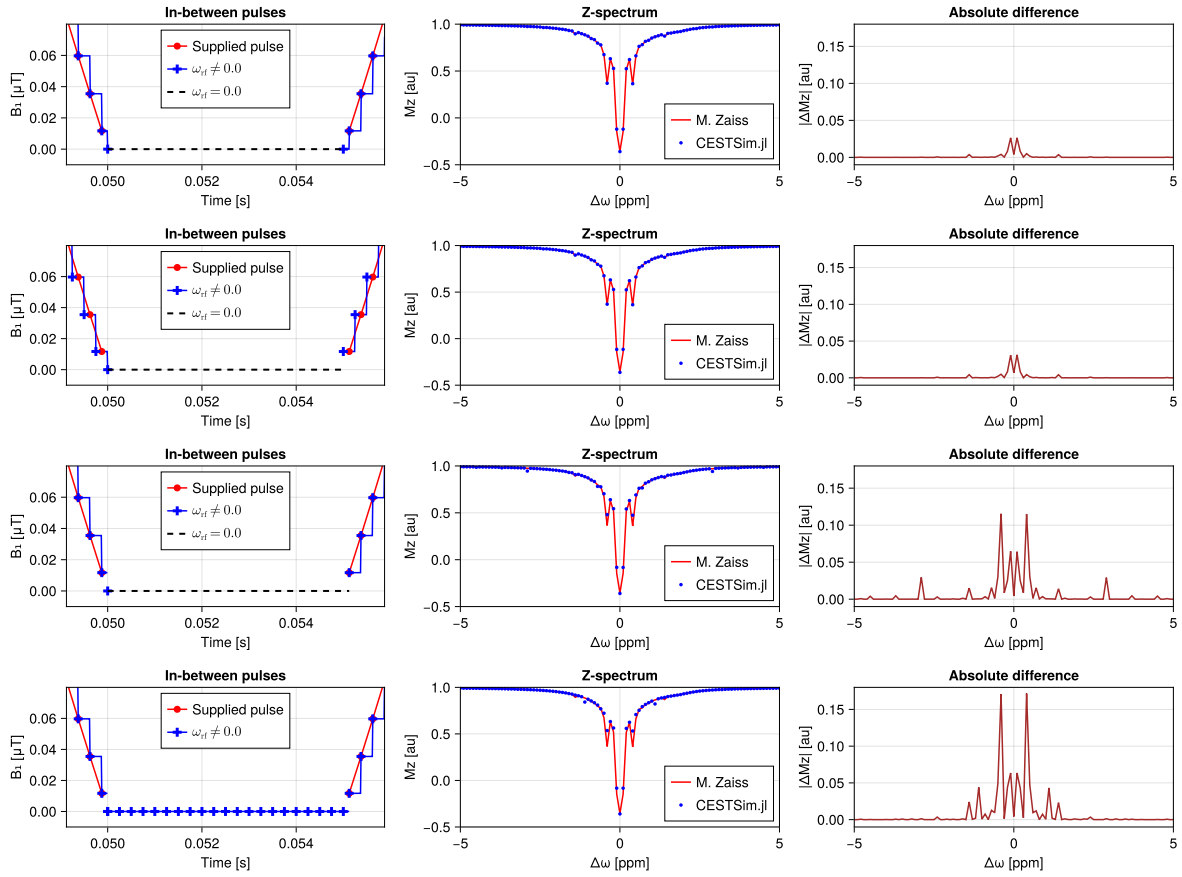



Figure 2: Influence of interpolation time grid on the RF frequency offset during interpulse delays. See text for implementation details. The top two rows show similar spectra and small differences to the reference (S01), while the bottom two rows show larger deviations.

## 3 Interpolation of the shaped RF pulse

Case 5 provides the Gaussian pulse as a sampled waveform (time and amplitude in a CSV file). Solvers that do not use those samples as-is must interpolate onto their own time grid. Figure 3 compares three common interpolation methods, nearest-neighbor, linear, and piecewise cubic Hermite (PCHIP), applied in submission S03, with Pulseq-CEST (S01) as the reference.

On the scale of the $Z$-spectrum and $MTR_{\mathrm{asym}}$, the four traces overlay. Point-wise differences relative to Pulseq-CEST remain below about $3 \times 10^{-4}$ in $Z$ (largest for linear interpolation near 0 ppm) and below about $6 \times 10^{-8}$ in $MTR_{\mathrm{asym}}$. Nearest-neighbor interpolation stays closest to the reference in $Z$ ($\sim 2 \times 10^{-5}$ near water). These differences are much smaller than the Case 5 spread among challenge submissions (maximum $Z$ spread $3.8 \times 10^{-3}$), so interpolation does not explain the Case 5 spread, but it is a source of small systematic discrepancy and should be reported.

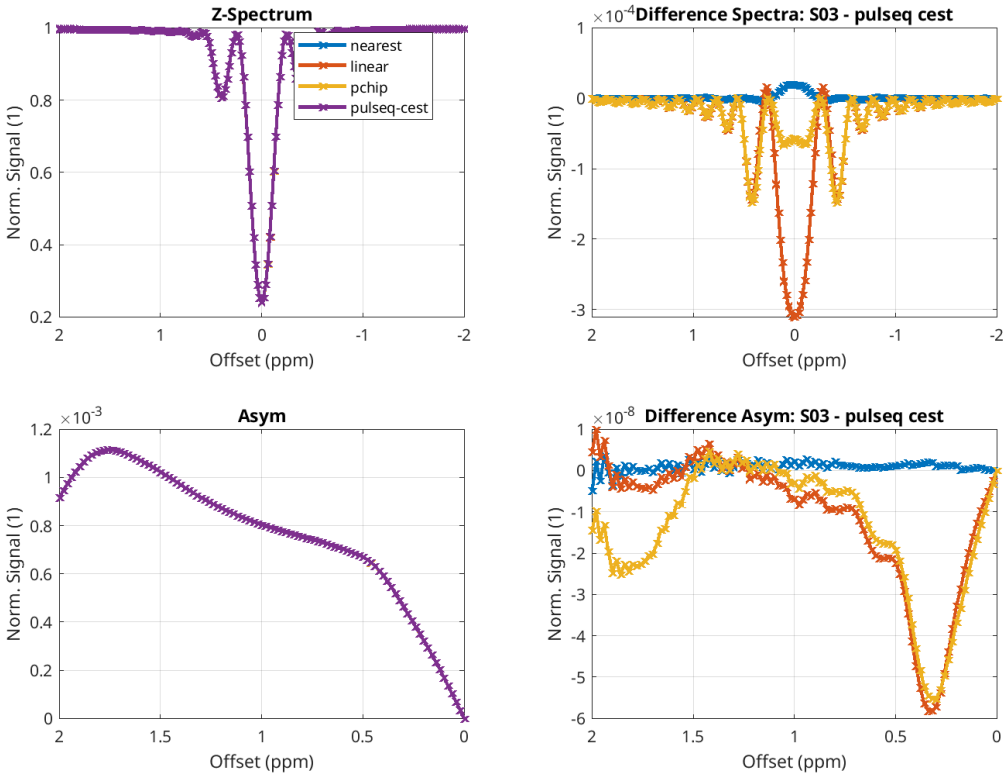



Figure 3: Case 5 $Z$-spectra and $MTR_{\mathrm{asym}}$ for a single Gaussian pulse simulated with nearest-neighbor, linear, and PCHIP interpolation of the sampled RF waveform in S03, compared with Pulseq-CEST (S01). Difference panels show S03 minus Pulseq-CEST.

# 4 Numerical precision and time-step convergence

Two numerical checks described in the main manuscript (Sections 2.7 and 3.4) are shown in Figure 4. For the BART submission, single- (S14) and double-precision (S14b) variants were compared with the reference S02 in Cases 1 and 2. For the operator-splitting MATLAB submission S03, Case 7 $Z$-spectra were calculated with fixed timesteps from 500 to 5 $\mu$s.

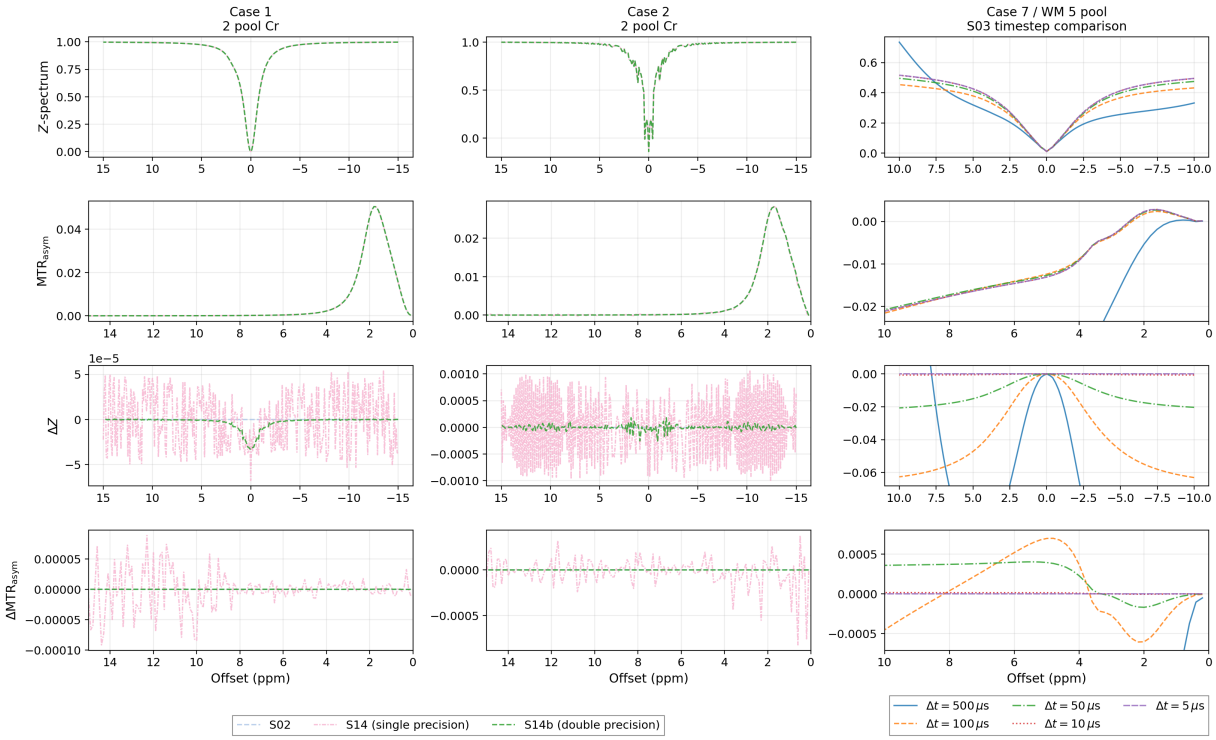



Figure 4: Numerical implementation checks. Left and center: single- and double-precision BART simulations (S14 and S14b) compared with the reference S02 for Cases 1 and 2; difference panels show deviations from S02. Right: Case 7 $Z$- and $MTR_{\mathrm{asym}}$ spectra for submission S03 simulated with fixed timesteps from 500 to 5 $\mu$s; difference panels show deviations from the finest timestep (5 $\mu$s). Axis limits in the Case 7 difference panels were chosen to highlight agreement among the smaller timesteps.

# References


[1] Herz Kai, Mueller Sebastian, Perlman Or, et al. Pulseq-CEST: Towards multi-site multi-vendor compatibility and reproducibility of CEST experiments using an open-source sequence standard. *Magnetic Resonance in Medicine.* 2021;86:1845-1858.

[2] Graf Christina, Rund Armin, Aigner Christoph Stefan, Stollberger Rudolf. Accuracy and performance analysis for Bloch and Bloch-McConnell simulation methods. *Journal of Magnetic Resonance.* 2021;329:107011.

[3] Stilianu Clemens, Huemer Markus, Zaiss Moritz, Stollberger Rudolf. Generalization of Optimal Control Saturation Pulse Design for Robust and High CEST Contrast. *Magnetic Resonance in Medicine.* 2025;.

[4] Huemer Markus, Stilianu Clemens, Scholand Nick, et al. Dynamic Transitions for Fast Joint Acquisition and Reconstruction of CEST-Rex and T1. *Magnetic Resonance in Medicine.* 2025;.

[5] Vladimirov Nikita, Cohen Ouri, Heo Hye Young, Zaiss Moritz, Farrar Christian T., Perlman Or. Quantitative molecular imaging using deep magnetic resonance fingerprinting. *Nature Protocols 2025 20:10.* 2025;20:3024-3054.

[6] Zaiss Moritz, Angelovski Goran, Demetriou Eleni, McMahon Michael T., Golay Xavier, Scheffler Klaus. QUESP and QUEST revisited – fast and accurate quantitative CEST experiments. *Magnetic Resonance in Medicine.* 2018;79:1708-1721.

[7] Blumenthal Moritz, Schaten Philip, Mackner Daniel, et al. *mrirecon/bart: version 1.0.00.* 2026.

[8] Scholand Nick, Wang Xiaoqing, Roeloffs Volkert, Rosenzweig Sebastian, Uecker Martin. Quantitative MRI by nonlinear inversion of the Bloch equations. *Magnetic Resonance in Medicine.* 2023;90:520-538.

[9] Steinebach Gerd. Construction of Rosenbrock–Wanner method Rodas5P and numerical benchmarks within the Julia Differential Equations package. *BIT Numerical Mathematics 2023 63:2.* 2023;63:27-.

[10] Zimmermann Felix Frederik, Schuenke Patrick, Aigner Christoph S., et al. MRpro - open PyTorch-based MR reconstruction and processing package. 2025;.